

A Placement-Based Deep Q-Learning Agent for Tetris: Afterstate Value Estimation and Reward Shaping

Author Name

Department / Program, Institution

email@example.com

Abstract—Tetris is a classic yet computationally hard benchmark for reinforcement learning, combining a vast state space with sparse, delayed reward. We present a placement-based Deep Q-Learning (DQN) agent that, instead of acting on low-level controls, reasons over *afterstates*: for each tetromino it enumerates all legal final placements, summarises the resulting board with four interpretable features (lines cleared, holes, bumpiness, and aggregate height), and uses a compact two-layer multilayer perceptron to estimate placement value. We study two reward functions on this architecture: a sparse score-based baseline and a dense, potential-style shaped reward that penalises holes, surface roughness, and height while rewarding survival. Trained for 2,000 epochs under the baseline reward, the agent improves from essentially random play to a mean of ≈ 167 cleared lines per game, peaking at 803 lines and a score of 20,722. We analyse how the shaped reward alters credit assignment and the optimisation landscape, and discuss the trade-off between optimisation ease and objective fidelity.

Keywords—Reinforcement learning, Deep Q-Network, Tetris, afterstates, reward shaping, value function approximation.

I. INTRODUCTION

Tetris is one of the most studied benchmarks in sequential decision making. Despite simple rules—rotate and translate falling tetrominoes to complete and clear horizontal lines on a 20×10 grid—the game is provably hard: maximizing the number of cleared lines is NP-complete and even hard to approximate [4]. The combination of a very large state space (every board configuration), stochastic piece arrivals, long horizons, and sparse, delayed reward (lines clear only when a row is completely filled) makes Tetris an attractive testbed for reinforcement learning (RL).

This paper describes a placement-based Deep Q-Learning agent for Tetris. Rather than learning over the low-level controls (left, right, rotate, drop), the agent reasons over *afterstates*: for each new tetromino it enumerates every legal final placement, computes a compact four-dimensional feature description of the resulting board, and uses a small neural network to estimate the value of each candidate placement [10]. This afterstate formulation collapses the branching of intermediate moves and is known to outperform raw state-action value functions on Tetris [1, 3].

A central practical question for such agents is *reward design*. The nominal game score rewards line clears but gives the agent almost no signal for the long stretches of play in which no line is cleared. We therefore study two reward functions on the same architecture: (i) a *baseline* reward derived from the game score, and (ii) a *shaped* reward that adds dense, potential-style penalties on holes, surface roughness (bumpiness), and stack height, together with a small survival bonus and a large terminal penalty. We report the empirical learning dynamics of the baseline agent over 2,000 training epochs and analyse how the shaped reward changes the optimisation landscape and credit assignment.

Contributions.

- A complete, reproducible placement-based DQN for Tetris built on a four-feature afterstate representation and a compact multilayer perceptron.

- A side-by-side formulation and analysis of two reward functions—a sparse score-based baseline and a dense shaped reward—highlighting the trade-offs each induces.
- An empirical study of the baseline agent showing that mean episode score rises from ≈ 17 to $\approx 3,900$ and mean cleared lines per game from near zero to ≈ 167 within 2,000 epochs, with a peak of 803 lines cleared in a single game.

II. RELATED WORK

Classical feature-based value functions. The earliest successful Tetris controllers used linear value functions over hand-crafted board features. Tsitsiklis and Van Roy applied feature-based value iteration using board height and the number of holes [12], and Bertsekas and Tsitsiklis popularised neurodynamic programming on Tetris [3]. Later refinements with richer feature sets and better optimisation—culminating in the cross-entropy and modified policy iteration results of Gabillon *et al.*—cleared on the order of millions of lines, establishing that careful features plus value-based control are remarkably effective on this domain [6]. Algorta and Şimşek survey this literature and emphasise the importance of the afterstate formulation [1].

Deep reinforcement learning. Deep Q-Networks (DQN) demonstrated that a single neural architecture trained with experience replay and a target network can reach human-level control on Atari games directly from pixels [9], building on Q-learning [14], experience replay [8], and later stabilised by Double DQN [13]. Applying end-to-end DQN to raw Tetris pixels is difficult because of the extreme reward sparsity; most successful deep agents therefore combine deep value estimation with the classical afterstate-plus-features trick.

Deep Tetris agents. Several open implementations follow this hybrid recipe. Nguyen’s PyTorch agent learns a value function over four board features and serves as the basis for the environment used here [10]. Anlay [2] and Faria [5] report similar feature sets (cleared lines, holes, bumpiness, aggregate height)

and strong play. Student projects by Stevens and Pradhan [11] and the high-level-state DQN study of Lewis *et al.* [7] explicitly investigate alternative reward functions, motivating the reward comparison we conduct.

III. BACKGROUND

A. Markov Decision Process

We model Tetris as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where a state s encodes the board occupancy and the current piece, $\mathcal{A}(s)$ is the set of legal placements, P captures the (deterministic) board update plus the stochastic arrival of the next piece, r is the reward, and $\gamma \in [0, 1)$ discounts future reward. The agent seeks a policy π maximising the expected return $\mathbb{E}[\sum_{t \geq 0} \gamma^t r_t]$.

B. Q-Learning and Afterstates

Q-learning [14] learns the optimal action-value function $Q^*(s, a)$ via the Bellman update. In Tetris, the transition decomposes naturally: once a placement is chosen, the resulting board (the *afterstate*) is deterministic; only the next piece is random. It is therefore efficient to learn a value function $V_\theta(\phi)$ over the *features* ϕ of the *afterstate*, and at decision time pick the placement whose afterstate has the highest value:

$$a^* = \arg \max_{a \in \mathcal{A}(s)} V_\theta(\phi(\text{place}(s, a))). \quad (1)$$

This sidesteps the large state–action space of low-level controls and yields a single-output network, which is the design adopted here.

IV. METHOD

A. Environment

The environment is a standard Tetris board of height 20 and width 10 with the seven canonical tetrominoes (I, O, T, S, Z, J, L) drawn from a shuffled 7-bag randomiser, so each piece appears once per bag. The board is rendered at a block size of 20 px for visualisation and video capture. A move consists of selecting a target column and rotation; the piece then hard-drops, locks, completed rows are cleared, and a new piece is spawned. The episode ends when a newly spawned piece collides immediately (top-out).

B. Afterstate Action Enumeration

For the current piece the environment enumerates all legal placements. The number of distinct rotations is piece-dependent to avoid duplicates: the O piece has 1, the S, Z and I pieces have 2, and the T, J and L pieces have 4 rotations. For each rotation the agent sweeps every horizontal offset that keeps the piece inside the board, simulates the hard drop, writes the piece to a copy of the board, clears completed rows, and records the feature vector of the resulting afterstate. This yields a dictionary mapping each candidate placement $(x, \text{rotation})$ to its feature vector.

C. State Features

Each afterstate is summarised by four interpretable features computed on the post-placement board:

TABLE 1. AFTERSTATE FEATURE VECTOR $\phi(s') \in \mathbb{R}^4$.

Symbol	Description	Tendency
ℓ	Lines cleared by the placement	maximise
h	Number of holes	minimise
b	Bumpiness (adjacent height diffs)	minimise
H	Aggregate column height	minimise

- **Lines cleared** (ℓ): rows completed by this placement.
- **Holes** (h): empty cells that have at least one filled cell somewhere above them in the same column.
- **Bumpiness** (b): the sum of absolute height differences between adjacent columns, measuring surface roughness.
- **Aggregate height** (H): the sum of the per-column heights.

These four features (Table 1) are the de-facto standard for feature-based Tetris and are sufficient for strong play [2, 5, 12].

D. Network Architecture

The value function $V_\theta : \mathbb{R}^4 \rightarrow \mathbb{R}$ is a compact multilayer perceptron with two hidden layers of 64 units and ReLU activations, followed by a single linear output:

$$\mathbb{R}^4 \xrightarrow{\text{FC}_{64} + \text{ReLU}} \mathbb{R}^{64} \xrightarrow{\text{FC}_{64} + \text{ReLU}} \mathbb{R}^{64} \xrightarrow{\text{FC}_1} \mathbb{R}.$$

The network has roughly 4,500 parameters, making both training and inference inexpensive. Two copies are maintained: an online network V_θ and a periodically synchronised target network V_{θ^-} .

E. Deep Q-Learning Procedure

Training follows the DQN template [9] adapted to afterstate values. At each step the agent enumerates candidate afterstates and selects one ε -greedily: with probability ε it explores a random legal placement, otherwise it chooses the placement of highest predicted value (Eq. 1). The transition—current afterstate features, reward, next afterstate features, and a done flag—is pushed into a replay buffer. Mini-batches are sampled uniformly [8], and the network is regressed onto the one-step bootstrap target

$$y = r + \gamma(1 - \text{done})V_{\theta^-}(\phi'), \quad (2)$$

by minimising the squared temporal-difference error $(y - V_\theta(\phi))^2$ with the Adam optimiser. The exploration rate ε is annealed linearly from 1.0 to 10^{-3} over the course of training. Table 2 lists the hyperparameters; the pseudocode is given in Fig. 1.

F. Reward Functions

We compare two reward designs that share the architecture and training loop.

Baseline (score-based). The reward equals the incremental game score,

$$r_{\text{base}} = 1 + w \cdot \ell^2, \quad (3)$$

TABLE 2. TRAINING HYPERPARAMETERS.

Hyperparameter	Symbol	Value
Discount factor	γ	0.99
Learning rate (Adam)	η	1×10^{-3}
Initial exploration	ϵ_0	1.0
Final exploration	ϵ_T	1×10^{-3}
Exploration decay horizon	—	2,000 epochs
Replay buffer capacity	—	30,000
Mini-batch size	—	512
Hidden units / layer	—	64
Training epochs	—	2,000

Algorithm 1: Afterstate DQN for Tetris

```

init  $V_\theta$ , target  $V_{\theta^-} \leftarrow V_\theta$ , replay  $\mathcal{D}$ 
for epoch = 1... $N$  do
  reset board; obtain features  $\phi$ 
  while not game over do
     $\{(\phi_a)\} \leftarrow$  enumerate legal afterstates
    with prob.  $\epsilon$ : pick random  $a$ ;
    else  $a \leftarrow \arg \max_a V_\theta(\phi_a)$ 
    execute  $a$ ; observe  $r, \phi', \text{done}$ 
    store  $(\phi, r, \phi', \text{done})$  in  $\mathcal{D}$ ;  $\phi \leftarrow \phi'$ 
    sample batch;  $y = r + \gamma(1 - \text{done})V_{\theta^-}(\phi')$ 
    gradient step on  $(y - V_\theta(\phi))^2$ 
  decay  $\epsilon$ ; periodically  $\theta^- \leftarrow \theta$ 

```

Fig. 1. Placement-based DQN training loop.

where $w = 10$ is the board width and ℓ is the number of lines cleared by the move; a terminal penalty of -2 is applied on top-out. The $+1$ per placed piece rewards survival, while the quadratic term strongly favours multi-line clears (a single line yields 11, a Tetris of four lines yields 161). This signal is, however, almost entirely flat for the many placements that clear no line.

Shaped (potential-style). The second reward adds dense feedback from the change in board quality caused by the placement:

$$r_{\text{shape}} = 40\ell - 8\Delta h - 2\Delta b - 1\Delta H - 5 \cdot \mathbb{I}[H_{\max} \geq 16] + 0.1, \quad (4)$$

where $\Delta h, \Delta b, \Delta H$ are the changes in holes, bumpiness, and aggregate height between the board before and after the placement, $H_{\max}$ is the tallest column, and the trailing $+0.1$ is a per-step survival bonus. A top-out incurs a large terminal penalty of -100 . Unlike Eq. (3), every placement now produces an informative reward: creating holes or a jagged, tall surface is penalised immediately rather than only through the eventual game-over. The height-threshold term provides an explicit safety margin against topping out.

V. EXPERIMENTAL SETUP

The agent was trained for 2,000 epochs (one epoch = one full game) under the baseline reward of Eq. (3). We logged,

TABLE 3. PERFORMANCE AT TRAINING MILESTONES (MEANS OVER 100-EPOCH WINDOWS).

Epoch window	Mean score	Mean lines	Mean pieces
1–100	17	0.1	19
501–600	21	0.2	21
1001–1100	34	0.8	27
1501–1600	151	8.5	53
1901–2000	3,947	166.9	449

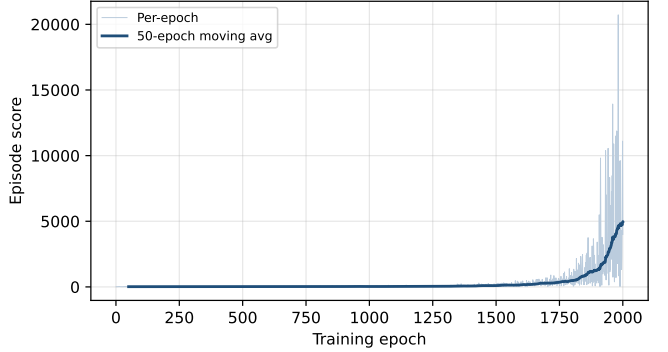


Fig. 2. Episode score per training epoch (light) with a 50-epoch moving average (dark). After a long warm-up, performance accelerates sharply once the value function matures and ϵ becomes small.

per epoch, the total episode score, the number of cleared lines, the number of placed tetrominoes (a proxy for survival length), the exploration rate ϵ , and the training loss, using TensorBoard. The final policy and target networks, the optimiser state, and the exploration rate were checkpointed for evaluation and replay. All curves below are computed directly from these logs; per-epoch traces are shown together with a 50-epoch moving average to expose the trend through the noise inherent in a stochastic game.

VI. RESULTS

A. Learning Dynamics

Fig. 2 shows the episode score over training. Learning exhibits a long, slow warm-up followed by a sharp take-off. As summarised in Table 3, the mean score stays near 17–34 for the first $\sim 1,100$ epochs while ϵ is still high and the value function is immature, climbs to ≈ 151 around epoch 1,500, and then rises steeply to a mean of $\approx 3,947$ over the final 100 epochs. The best single game reached a score of 20,722.

B. Line-Clearing Ability

The number of cleared lines per game (Fig. 3) follows the same pattern. The agent clears essentially no lines for the first thousand epochs, then improves rapidly, averaging ≈ 167 cleared lines per game over the final 100 epochs and peaking at 803 lines in a single game. Because line clears are the only source of large baseline reward, this curve tracks the score curve closely and confirms that the policy learned the intended

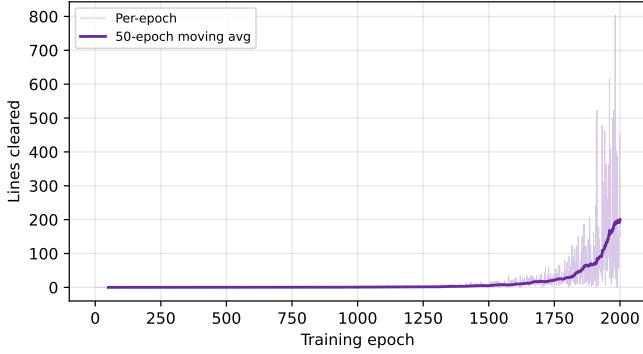


Fig. 3. Lines cleared per game. The agent transitions from near-zero clears to hundreds of lines per game, peaking at 803.

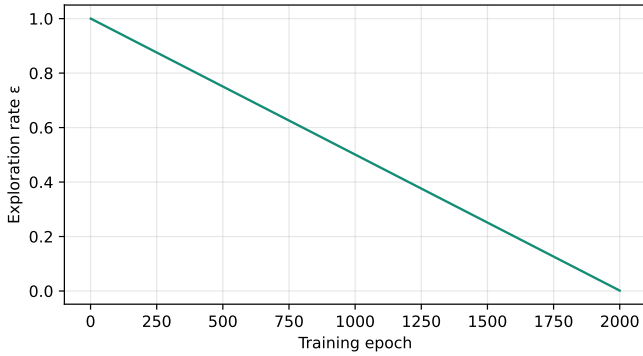


Fig. 4. Exploration rate ϵ annealed linearly from 1.0 to 10^{-3} over 2,000 epochs.

behaviour—deliberately completing rows—rather than merely surviving.

C. Exploration and Optimisation

Fig. 4 shows the linear annealing of ϵ from 1.0 to 10^{-3} . The performance take-off coincides with the late stage of this schedule, when the agent increasingly follows its own value estimates. Fig. 5 shows the temporal-difference loss on a logarithmic scale. The loss does *not* decrease monotonically: it grows as the agent begins to survive longer and to encounter much larger returns, which enlarges the regression targets. This is expected and benign for value-based control on a task whose return magnitude itself increases with competence; the policy continues to improve even as the raw loss rises.

D. Reward-Design Analysis

The two reward functions differ most in their *density* and *credit assignment*. The baseline reward of Eq. (3) is sparse: away from line clears it returns a constant +1, so the gradient signal that distinguishes a clean, flat board from one riddled with holes arrives only indirectly, through the eventual top-out penalty propagated over many steps. This explains the very long warm-up observed in Figs. 2–3: the agent must accumulate enough experience for bootstrapping to back-propagate the

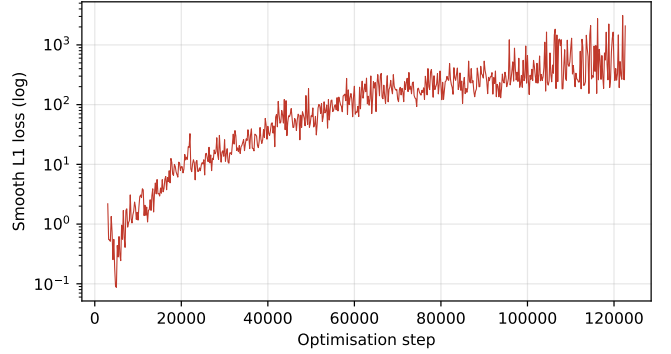


Fig. 5. Temporal-difference regression loss (log scale). The loss rises as episodes lengthen and returns grow, which is typical for value-based agents on tasks with competence-dependent return magnitude.

consequences of poor placements. The shaped reward of Eq. (4) instead delivers an informative signal on *every* placement—penalising new holes (-8 each), added roughness (-2 per unit), and growth in height (-1 per unit), with an explicit safety penalty near the top of the board and a large -100 terminal cost. By converting long-horizon credit assignment into immediate feedback, the shaped reward is expected to shorten the warm-up and stabilise early learning; its principal risk is bias—hand-tuned weights can steer the agent toward locally tidy boards that are not globally score-optimal, and the large terminal penalty must be balanced against the per-step bonus to avoid overly conservative play. The baseline reward, being closer to the true game objective, is less biased but harder to optimise. This trade-off between optimisation ease (dense shaping) and objective fidelity (sparse score) is the central design tension for Tetris RL [7, 11].

VII. DISCUSSION

Three observations stand out. First, the four-feature afterstate representation, despite its extreme compactness, is sufficient for a tiny MLP to reach strong play—hundreds of lines per game—corroborating the classical finding that good features matter more than model capacity on Tetris [6, 12]. Second, the learning curve is highly non-linear: most of the apparent progress is concentrated in the final quarter of training, a regime that only becomes reachable once exploration has decayed and the value estimates are accurate enough for the $\arg \max$ in Eq. (1) to be reliable. Practitioners should therefore avoid judging such agents by their early epochs. Third, reward shaping is a lever with real consequences: it can dramatically ease optimisation but introduces bias that must be validated against the true objective.

Limitations. The empirical results reported here are for the baseline reward; a full quantitative comparison of the shaped reward under identical seeds and budgets is the natural next step. Results are from a single training run, so confidence intervals over multiple seeds would strengthen the claims. Finally, the agent observes only four engineered features, so it cannot exploit board structure invisible to those features (e.g. specific

well shapes).

VIII. CONCLUSION AND FUTURE WORK

We presented a placement-based Deep Q-Learning agent for Tetris that learns a value function over four interpretable after-state features with a compact two-layer MLP. Trained for 2,000 epochs under a score-based reward, the agent improves from essentially random play to a mean of ≈ 167 cleared lines per game, peaking at 803 lines and a score of 20,722. We further formulated and analysed a dense, potential-style shaped reward and discussed the optimisation-versus-fidelity trade-off it introduces relative to the sparse baseline. Future work includes a controlled empirical comparison of the two rewards across multiple seeds, prioritised experience replay and Double-DQN targets [13] to stabilise the value regression, richer features or a convolutional encoder over the raw board, and curriculum or self-play schemes to shorten the lengthy warm-up phase.

REFERENCES

- [1] Simon Algorta and Özgür Şimşek. The game of Tetris in machine learning. *arXiv preprint arXiv:1905.01652*, 2019.
- [2] Andrian Anlay. Tetris-AI: Deep rl Tetris agent using value-function learning and hand-crafted features. <https://github.com/andriananlay/tetris-ai-deep-reinforcement-learning>, 2020. GitHub repository.
- [3] Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, Belmont, MA, 1996.
- [4] Erik D. Demaine, Susan Hohenberger, and David Liben-Nowell. Tetris is hard, even to approximate. *Int. J. Computational Geometry & Applications*, 14(1-2):41–68, 2004.
- [5] Nuno Faria. tetris-ai: A deep reinforcement learning bot that plays Tetris. <https://github.com/nuno-faria/tetris-ai>, 2020. GitHub repository.
- [6] Victor Gabillon, Mohammad Ghavamzadeh, and Bruno Scherrer. Approximate dynamic programming finally performs well in the game of Tetris. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.
- [7] Joseph Lewis, Justin Beckwith, et al. Applying deep Q-networks (DQN) to the game of Tetris using high-level state spaces and different reward functions. Technical Report, ResearchGate, 2020.
- [8] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3):293–321, 1992.
- [9] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [10] Viet Nguyen. Tetris-deep-Q-learning-pytorch. <https://github.com/uvipen/Tetris-deep-Q-learning-pytorch>, 2020. GitHub repository.
- [11] Matt Stevens and Sabeek Pradhan. Playing Tetris with deep reinforcement learning. In *Stanford University CS231n Course Project Report*, 2016.
- [12] John N. Tsitsiklis and Benjamin Van Roy. Feature-based methods for large scale dynamic programming. *Machine Learning*, 22:59–94, 1996.
- [13] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proc. AAAI Conf. on Artificial Intelligence*, volume 30, 2016.
- [14] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, 1992.